

קורס NodeJS תשפו

MongoDB

ככל שפת server, בnodeJS קיימים drivers להתחברות לDBs שונים. MongoDB, databases שעובד על JSON objects, הוא בחירה טבעית לשימוש עם nodeJS היות וJSON objects הם חלק טבעי מjavascript וניתן לנהל אותם בקלות רבה.

כך, בשליפת document מmongo, הdocument הנקרא מוחזק בobject רגיל של javascript – כשבמקרה של typescript הוא יקבל type שנגדיר לו. באותו אופן גם בהוספת document לcollection בmongo – הobject של javascript שבtypescript יכול לייצג type, נוסף לcollection בקלות.

הdriver שנשתמש בו כדי לבצע שאילתות לmongoDB הוא mongodb – החבילה הרשמית של mongo.

התקנה

```
npm i mongodb
```

חיבור לCluster

כדי להתחבר לDB cluster נגדיר משתנה שיכיל את connection string. אם connection string מכיל את כל הפרטים, כולל המשתמש והסיסמא, החיבור יוכל להתבצע באמצעותו מבלי הגדרות נוספות.

```
import { MongoClient } from "mongodb";

const DB_URL = "mongodb://admin:password@cluster0.mongodb.net";

export default class DbConnect {
  private mongoClient!: MongoClient;
  constructor(private url = DB_URL) {}

  public async init() {
    this.mongoClient = await MongoClient.connect(this.url);
  }
}
```

הפונקציה connect מקבלת פרמטר שני של options, בו ניתן להעביר הגדרות שונות – מהגדרות הנוגעות לתהליך ההתחברות עצמו כמו auth, ssl, connectTimeout ועד הגדרות שישפיעו על כל הפעולות שיתבצעו מול הDB באמצעות החיבור הנוצר – כמו maxPoolSize (מגדיר כמה connections ניתן לפתוח בו זמנית מול DB) ועוד.

חיבור לDB

לאחר יצירת החיבור הבסיסי לcluster, ניתן להתחבר לDB עצמו שקיים בתוך cluster. אם הDB המבוקש לא קיים, הוא ייוצר באופן אוטומטי.

```
public getStoreDb(dbName: string = 'store'): Db
{
    return this.mongoClient.db(dbName);
}
```

הפונקציה db() מקבלת כפרמטר שני db options, שמכיל configurations הנוגעים לDB החדש שיווצר במידה ולא קיים.

בסיום הרצת הפרויקט, יש לסגור את החיבור לcluster באמצעות קריאה ל:mongoClient.close()

```
public async terminate() {
    await this.mongoClient.close();
}
```

חיבור לCollection

לאחר החיבור לcluster ולdb, ניתן להשתמש באובייקט Db כדי לגשת לcollections בDB הרצוי.

כשמדברים על קוד typescript שמארגן בשכבות, החיבור לcollection יתבצע בד"כ בשכבת dal של resource collection.

ראשית, יש להגדיר את model של resource בDB, לדוגמא:

```
export type Product = {
    code: number;
    name: string;
    price: number;
}
```

כעת ניתן להתחבר לcollection, תוך הגדרת הtype של הcollection בעזרת המodel שהגדרנו:

```
const PRODUCTS_COLLECTION_NAME = 'products';

export default class ProductsDal {
  private collection!: Collection<Product>;
  constructor(private dbConn: DbConnect) {}

  public async init() {
    const db = this.dbConn.getStoreDb();
    this.collection =
db.collection<Product>(PRODUCTS_COLLECTION_NAME);
  }
}
```

כמו בהתחברות לבד, גם בהתחברות לcollection אם לא קיים collection בשם המבוקש, הוא ייוצר באופן אוטומטי. הגדרות הנוגעות ליצירת הcollection ניתן להעביר כפרמטר שני לפונקציה db.collection.

CRUD Operations

לאחר ההתחברות לcollection, ניתן להשתמש באובייקט collection כדי לבצע את כל פעולות הCRUD על הcollection – Create, Read, Update, Delete.

Create

הוספת document חדש:

```
public async addProduct(product: Product) {
  await this.collection.insertOne(product);
}
```

ניתן גם להוסיף documents רבים בו זמנית:

```
public async addMultipleProducts(products: Array<Product>) {
  await this.collection.insertMany(products);
}
```

שליפת כל documents מהcollection:

```
await this.collection.find().toArray();
```

האובייקטים שיחזרו מהפונקציה, יכילו את כל השדות של Product בתוספת השדה של `_id` – הקיים באופן אוטומטי על כל document בmongo. לכן, הtype של כל אובייקט במערך יהיה: `WithId<Product>`, כאשר `WithId` מגדיר שהtype הוא בתוספת השדה `_id`.

הפונקציה הבאה מדגימה כיצד ניתן להמיר את האובייקטים לtype הרגיל של resource ע"י הסרת השדה `_id`:

```
public async getAllProducts(): Promise<Array<Product>> {
    const productsFromDb: Array<WithId<Product>> = await
    this.collection.find().toArray();
    return productsFromDb.map(product => ({...product, _id:
    undefined}));
}
```

הפונקציה `find()` מקבלת filter, כך שניתן לבצע שאילתות לDB ולשלוף documents לפי קריטריונים. לדוגמא, שליפת כל products שמחירם גבוה מ50:

```
await this.collection.find({price: {$gt: 50}}).toArray();
```

על גבי תוצאת הfind, לפני ההמרה לarray, ניתן לקרוא לפונקציות נוספות ולהרחיב את query, לדוגמא:

- **project** – שליפת שדות מסוימים מתוך documents שנמצאו.
- **sort** – מיון documents לפי פרמטרים.
- **skip** – דילוג על documents בכמות מוגדרת והחזרת תוצאות רק מהdocuments שנותרו.
- **limit** – הגבלת כמות documents שיחזרו.

כל הפונקציות הנ"ל, כולל פונקציית הfind עצמה, פועלות באופן דומה מאד לqueries רגילים הרצים על mongo בעזרת mongo shell, כך שניתן להריץ את אותם queries כולל כל האופציות והשלבים.

כמו כן, הפונקציה aggregate זמינה גם היא על אובייקט הcollection ופועלת באותה צורה של הפונקציה בmongo shell – מקבלת מערך של stages ומריצה אותם בזה אחר זה.

שליפת document בודד:

```
public async getProduct(code: number): Promise<Product | null> {
    const productFromDb = await this.collection.findOne({code});
    if (!productFromDb) {
        return null;
    }

    const product = {...productFromDb, _id: undefined};
    return product;
}
```

Update

עדכון document בודד:

```
public async updateProduct(code: number, product: Partial<Product>)
{
    await this.collection.updateOne({code}, {$set: product});
}
```

עדכון documents רבים:

```
await this.collection.updateMany(
    { price: { $gt: 100 } },
    [{
        $set: {
            price: { $multiply: ["$price", 0.7] }
        }
    }]
);
```

ישנן פונקציות נוספות לעדכון documents, ביניהן:

- **findOneAndUpdate** – עדכון document בודד והחזרתו לאחר העדכון.
- **replaceOne** – החלפת document בחדש.

ככל הפונקציות שפועלות על collection, גם פונקציות updaten מקבלות options כפרמטר שני. בין הoptions האפשריים, פונקציות updaten מקבלות option של upsert. כשהוא true, הפונקציה תוסיף document חדש במידה והdocument המבוקש לא קיים. הdocument החדש יכיל את השדות שהועברו לעדכון בפונקציה. לדוגמא:

```
await this.collection.updateOne({code}, {$set: product}, {upsert:
true});
```

Delete

מחיקת document בודד:

```
public async deleteProduct(code: number) {  
    await this.collection.deleteOne({code});  
}
```

מחיקת documents רבים:

```
public async deleteProducts(codes: Array<number>) {  
    await this.collection.deleteMany({code: {$in: codes}});  
}
```

פונקציות נוספות

הפונקציות הבאות פועלות על אובייקט הcollection:

- **bulkWrite** – מקבל מערך של operations, ומבצע את כולם במקביל. הoperations הנתמכים כוללים את פעולות הinsert, update, delete על document אחד או רבים. פונקציה זו שימושית ביותר לביצוע כמות גדולה של פעולות ביעילות – כמו הוספת אלפי documents, בעזרת חלוקה לchunks ושליחתם לbulkWrite.
- **drop** – מחיקת collection.
- **countDocuments** – החזרת מספר הdocuments בcollection שתואמים filter מסוים.
- **distinct** – החזרת כל הערכים הקיימים בcollection תחת field מסוים, תוך שמירה על uniqueness – כל ערך יחזור רק פעם אחת.